AIQUOS 前端实现汇报

各位领导，我是前端负责工程师，用几分钟把实现方案和现状讲清楚。

一、选型

React 19 加 Vite，没有引 UI 组件库，没有上 TypeScript。视觉是深度定制的，组件库进来反而要去样式；类型安全的压力放在评分和数据层，用纯函数加契约测试兜底，比类型标注更硬。整个项目只有四个运行时依赖，字体本地打包，不依赖外网。依赖少，升级和供应链风险就小。

二、架构

hash 路由驱动单页应用，一个编排组件管理视图栈和转场，没有路由库也没有 Redux。核心决策是测评状态全部收在纯函数模块里，React 组件只管调用和渲染——测评状态机可以脱离浏览器直接跑测试，这是小团队保住质量的最大杠杆。二十分钟的测评流程（对话、客观题、实操三阶段）由一个状态机编排器调度三个独立阶段组件，作答证据随每次答题落草稿，刷新可续做，登录用户草稿同步服务端，换设备也能接续。

三、三个难点

立方体首页。外壳用 three.js 渲染成单个连续 3D 网格，但三块屏幕的内容仍是实时 DOM——每帧用同一套投影矩阵把 DOM 平面贴合到屏幕位置，视频能播、按钮能点，外壳转多少内容跟多少。WebGL 不可用自动退回 DOM 投影，不留白屏。

转场。所有转场先把当前画面冻结成静止帧，新视图确认可呈现后再播动画，全程不见黑帧。案例跑在只允许脚本的隔离 iframe 里，限三十帧，页面不可见即暂停。

测评工作台。客观题由服务端逐题下发自适应出题，最终计分也在服务端重算；AI 服务不可用时全链路有标注清楚的离线降级。中文输入法回车、Safari 私密模式存储被拒这些坑，都有统一处理并配了守卫测试防回归。

四、质量

385 个自动化测试一分多钟跑完，分三层：纯函数契约、源码守卫（扫描旧坑模式，出现即测试失败）、真实浏览器渲染冒烟。构建链只有 npm run build 一道工序。

五、自评

做得好的：依赖极简，断网、无 WebGL、无存储、无 AI 服务都有明确降级而不是白屏；核心逻辑与界面解耦，可测性好；视觉完成度高且受测试保护。

不足直说：还有几个千行级大组件没拆完；样式总量偏大没做按需加载；缺一套端到端自动化，现在靠手动 QA 脚本；可访问性只做了键盘导航，没过读屏。

下一步按优先级：继续拆大组件、引 Playwright 固化端到端、论坛案例库做代码分割、系统过一遍可访问性。汇报完毕，欢迎指正。